Grado en Ingeniería Informática

Sistemas distribuidos 25/26

Grupo 81

Práctica Final

«Servicio de envío de mensajes»

Jorge Adrian Saghin Dudulea — 100522257

Denis Loren Moldovan — 100522240

Profesor

Felix García

Tabla de contenidos

1. **Descripción del código** ... 2
   - 1.1. Arquitectura general ... 2
   - 1.2. Servidor (`server.c`) ... 2
     - 1.2.1. Recepción y dispatch de mensajes ... 2
     - 1.2.2. REGISTER y UNREGISTER ... 2
     - 1.2.3. CONNECT ... 2
     - 1.2.4. DISCONNECT ... 2
     - 1.2.5. SEND y SENDATTACH ... 3
     - 1.2.6. conn_deliver ... 3
     - 1.2.7. USERS ... 3
   - 1.3. Cliente (`commands.py`) ... 3
     - 1.3.1. Hilo de escucha (_listen_server) ... 3
     - 1.3.2. SENDATTACH ... 3
     - 1.3.3. GETFILE ... 3
     - 1.3.4. Normalización de mensajes ... 4
   - 1.4. Capa de persistencia (`users.c`) ... 4
   - 1.5. Servicio web (`src/ws/app.py`) ... 4
   - 1.6. Servidor RPC (`src/rpc/`) ... 5
2. **Compilación y ejecución** ... 6
   - 2.1. Dependencias ... 6
   - 2.2. Compilación ... 6
     - 2.2.1. Contenedor de Docker ... 6
   - 2.3. Ejecución ... 6
     - 2.3.1. Ejecución de clientes con Docker ... 7
3. **Batería de pruebas** ... 8
4. **Conclusiones** ... 12

1. Descripción del código

La práctica implementa un servicio de mensajería distribuido formado por cuatro componentes: el servidor en C, el cliente en Python, un servicio web con Flask y un servidor de logging mediante ONC-RPC. Los componentes se comunican a través de sockets TCP, salvo la transferencia de ficheros, que es directamente P2P entre clientes.

1.1. Arquitectura general

El protocolo entre cliente y servidor usa mensajes de texto delimitados por `#` y terminados en el byte nulo (`\0`). El servidor responde siempre con un byte de código de estado (`0` = éxito, `1/2/3` = tipos de fallo específicos), seguido opcionalmente de más cadenas. Cada operación abre una conexión TCP nueva, envía su mensaje y cierra, excepto CONNECT, que es especial.

El servidor acepta conexiones en un bucle principal y lanza un hilo POSIX (`pthread`) por cada una. Para evitar condiciones de carrera, todos los accesos a la base de datos están protegidos por un mutex global.

1.2. Servidor (`server.c`)

1.2.1. Recepción y dispatch de mensajes

La función `recv_msg` lee el mensaje byte a byte hasta encontrar el `\0` y lo divide por `#` en un array de campos. El tamaño del buffer está dimensionado para acomodar el mensaje más largo posible, que es SENDATTACH con todos los campos al máximo. Después, `handle_client` compara el primer campo con los nombres de operación conocidos y llama al manejador correspondiente.

1.2.2. REGISTER y UNREGISTER

`handle_register` llama a `user_add` y devuelve el código de resultado directamente. `handle_unregister` hace lo propio con `user_remove`. En caso de éxito, ambas invocan `rpc_log` para registrar la operación.

1.2.3. CONNECT

`handle_connect` registra al usuario en la base de datos con su IP y el puerto de escucha que ha indicado. Lo interesante es que, después de enviar la respuesta, espera 100 ms con `usleep` **antes** de intentar entregar los mensajes pendientes. Este retardo da tiempo al cliente a abrir su socket de escucha para no perder mensajes que lleguen inmediatamente tras el CONNECT.

1.2.4. DISCONNECT

`handle_disconnect` comprueba que la petición viene de la misma IP desde la que el usuario se conectó, comparando la IP guardada en la base de datos con la del socket entrante. Esto evita que un proceso externo desconecte a otro usuario.

1.2.5. SEND y SENDATTACH

Ambos manejadores funcionan igual: verifican que el destinatario existe (aunque esté desconectado), almacenan el mensaje en la base de datos con `msg_add` y responden al cliente con el identificador asignado. Luego intentan entregar el mensaje de inmediato mediante `conn_deliver`. Si la entrega falla (el receptor se desconectó), el mensaje queda pendiente en la base de datos hasta el próximo CONNECT.

`handle_send_attached` es idéntico a `handle_send` salvo que también lee el campo del nombre de fichero y lo pasa a `msg_add` y `conn_deliver`. Para SENDATTACH, `rpc_log` recibe además el nombre del fichero.

1.2.6. conn_deliver

Esta función centraliza la entrega de mensajes al receptor. Construye la cadena del protocolo adecuada: `SEND_MESSAGE#...` para mensajes de texto o `SEND_MESSAGE_ATTACH#...` para mensajes con fichero. Se conecta al socket de escucha del receptor y envía el mensaje. Si la conexión falla, marca al usuario como desconectado. Si tiene éxito, borra el mensaje de la base de datos con `msg_delete` y envía el ACK al emisor (`SEND_MESS_ACK` o `SEND_MESS_ATTACH_ACK` según el caso). El envío del ACK es de tipo best-effort: si el emisor ya no está conectado, simplemente se omite.

1.2.7. USERS

`handle_users` verifica que el usuario solicitante está conectado y devuelve la lista de usuarios conectados. Para cada uno, además del nombre, envía su IP y puerto de escucha en formato `nombre::IP::puerto`. Esta información es necesaria para que los clientes puedan iniciar transferencias P2P.

1.3. Cliente (`commands.py`)

1.3.1. Hilo de escucha (_listen_server)

Al ejecutar CONNECT, el cliente reserva un puerto libre con `bind("", 0)`, envía la petición al servidor indicando ese puerto y arranca un hilo en segundo plano. Este hilo abre un socket de escucha en ese puerto y queda bloqueado en un bucle de `accept` con un timeout de 10 segundos. Cuando llega una conexión, lee el mensaje y lo pasa a `_parse_incoming_message`, que identifica el tipo de notificación (mensaje de texto, mensaje con fichero, ACK de entrega) y lo muestra por consola. El timeout permite que el hilo compruebe periódicamente la variable `_terminate` para saber si debe salir del bucle.

1.3.2. SENDATTACH

El método `send_attach` verifica en primer lugar que la ruta del fichero sea absoluta antes de hacer nada. Si no lo es, devuelve un error directamente sin contactar al servidor. El protocolo enviado es `SENDATTACH#emisor#receptor#mensaje#fichero`.

1.3.3. GETFILE

La transferencia de ficheros no pasa por el servidor; es directamente de cliente a cliente. El proceso es el siguiente:

1. El cliente solicitante busca en su caché interna (`_connected_users_info`) la IP y el puerto del usuario que tiene el fichero. Si no lo encuentra, llama a USERS para actualizar la caché.
2. Crea un socket TCP en un puerto libre y arranca un hilo que quedará escuchando en ese puerto.
3. Conecta directamente al socket de escucha del otro cliente y le envía el mensaje `GETFILE#solicitante#fichero_remoto#mi_ip#mi_puerto`.
4. El otro cliente, al recibir este mensaje en su hilo de escucha, abre el fichero indicado y lo envía en bloques de 4096 bytes al socket del solicitante.
5. El hilo receptor va escribiendo los bloques en el fichero local hasta que la conexión se cierra.

Para obtener la IP local correcta (la que el sistema operativo usaría para llegar al servidor), se usa un truco habitual con sockets UDP: se crea un socket DGRAM, se llama a `connect` con la dirección del servidor sin enviar ningún dato, y se lee la IP local del socket resultante. Así el SO selecciona la interfaz de red adecuada automáticamente.

1.3.4. Normalización de mensajes

Antes de enviar cualquier SEND, el cliente hace una petición POST al servicio web en `http://servidor:3000/quitar-espacios` con el texto del mensaje. Si el servicio responde con código 200, usa el texto normalizado. Si el servicio no está disponible o responde con error, el mensaje original se envía sin modificar. Este comportamiento de fallback es importante para que el cliente siga funcionando aunque el servicio web no esté activo.

1.4. Capa de persistencia (`users.c`)

La base de datos SQLite tiene dos tablas. La tabla `users` almacena el nombre (clave primaria), el estado de conexión (0/1), la IP, el puerto de escucha y un contador de mensajes. La tabla `messages` almacena los mensajes pendientes de entrega con sus campos: identificador numérico, emisor, receptor, texto y nombre de fichero (NULL para mensajes sin adjunto).

Los identificadores de mensaje son por receptor: cuando llega un mensaje nuevo, se incrementa el contador del receptor en la base de datos y el valor resultante se usa directamente como identificador del mensaje. Este diseño garantiza que el incremento y la lectura del ID son atómicos respecto al mutex. Los identificadores se envuelven de vuelta a 1 cuando llegan al máximo de `unsigned int`.

1.5. Servicio web (`src/ws/app.py`)

El servicio está implementado con Flask y expone un único endpoint: `POST /quitar-espacios`. Recibe un JSON con el campo `cadena`, divide el texto en palabras y las vuelve a unir con un único espacio entre cada una, eliminando así cualquier secuencia de espacios en blanco redundante. Devuelve el resultado como texto plano con código 200. El servidor escucha en `0.0.0.0:3000` para ser accesible desde otros contenedores o máquinas.

1.6. Servidor RPC (`src/rpc/`)

La interfaz está definida en `log_rpc.x`. El programa tiene número de programa 100, versión 1, y ofrece un único procedimiento `registrar` que recibe una estructura con tres cadenas de hasta 256 caracteres: nombre de usuario, operación y nombre de fichero. El servidor imprime esta información por consola al recibir cada llamada.

La función `rpc_log` del cliente RPC lee la variable de entorno `LOG_RPC_IP`. Si no está definida o está vacía, no hace nada. Si está definida, crea un cliente RPC con `clnt_create`, llama al procedimiento remoto y destruye el cliente. Los fallos son siempre silenciosos para no interrumpir el flujo normal de la aplicación.

Los ficheros `log_rpc_clnt.c`, `log_rpc_svc.c` y `log_rpc_xdr.c` fueron generados con `rpcgen` a partir del fichero `.x` y no se han modificado. Se compilan con solo `-O3` (sin `-Wall -Wextra -Werror`) para evitar que los warnings del código generado automáticamente rompan la compilación.

2. Compilación y ejecución

2.1. Dependencias

- `libtirpc` y sus cabeceras (para ONC-RPC)
- `libpthread`
- `rpcbind` en ejecución (necesario para el servidor RPC)

2.2. Compilación

```
make
```

El comando `make` compila el servidor de mensajería y el servidor RPC. Los binarios quedan en `./build/`. El código fuente de SQLite está en `lib/sqlite/` y se compila por separado con flags más permisivas, ya que no es código nuestro y genera warnings con `-Wall -Wextra`.

2.2.1. Contenedor de Docker

Debido a problemas con el firewall en Linux, y la definición del enunciado de que los clientes deben funcionar con IPs distintas, se ha escrito un pequeño Dockerfile para ejecutar los clientes, dentro de la misma máquina, sin utilizar como dirección el `localhost`.

```
docker build -t practica_final_distribuidos_40-57 .
```

2.3. Ejecución

Los procesos deben arrancarse en el siguiente orden:

1. **Servidor RPC** (opcional; necesario solo si se quiere logging):

   ```
   ./build/log_rpc_server
   ```

2. **Servicio web** (en la misma máquina que el cliente):

   ```
   python3 src/ws/app.py
   ```

3. **Servidor de mensajería** (con o sin logging RPC):

   ```
   LOG_RPC_IP=<ip_servidor_rpc> ./build/server -p <puerto>
   # Sin RPC:
   ./build/server -p <puerto>
   ```

4. **Clientes** (uno por usuario):

   ```
   python3 src/clients/client.py -s <ip_servidor> -p <puerto>
   ```

Los comandos disponibles en el cliente son:

Comando	Descripción
`REGISTER <nombre>`	Registrar un nuevo usuario
`UNREGISTER <nombre>`	Eliminar un usuario registrado
`CONNECT <nombre>`	Conectarse al servicio
`DISCONNECT <nombre>`	Desconectarse
`SEND <destino> <mensaje>`	Enviar un mensaje de texto
`SENDATTACH <dest> <msg> <ruta>`	Enviar mensaje con fichero adjunto (ruta absoluta)
`USERS`	Ver usuarios conectados con IP y puerto
`GETFILE <usuario> <remoto> <local>`	Descargar un fichero de otro usuario
`QUIT`	Salir del cliente

2.3.1. Ejecución de clientes con Docker

Con la imagen de los clientes construida, al ejecutarlas, cada una con nombre distinto, van a tener asignadas una dirección distinta, corriendo tanto el servicio web, como el cliente para ejecutar los comandos. Para ello:

```
docker run -it --rm --name cliente<n> -e SERVER_IP="<ip a>" -e
SERVER_PORT="<puerto definido en el runtime del servidor>"
practica_final_distribuidos_40-57
```

Y de fondo, para tener el servicio completo:

```
./build/server -p <puerto> && ./build/log_rpc_server
```

3. Batería de pruebas

Para probar el sistema arrancamos el servidor en una terminal, abrimos otra para el cliente y fuimos ejecutando los comandos a mano, comprobando que las respuestas que aparecían por pantalla coincidían con lo esperado. Repetimos esto para los distintos escenarios: registros, conexiones, envíos en línea y en diferido, adjuntos, transferencia de ficheros y el servidor RPC. En total ejecutamos 133 casos repartidos en 18 bloques, y todos dieron el resultado correcto. En la siguiente tabla recogemos los más representativos agrupados por funcionalidad.

Operación	Caso de prueba	Resultado esperado	Resultado obtenido
REGISTER	Registrar un usuario nuevo	`c> REGISTER OK`	Correcto
REGISTER	Registrar un nombre ya existente	`c> USERNAME IN USE`	Correcto
UNREGISTER	Eliminar un usuario existente	`c> UNREGISTER OK`	Correcto
UNREGISTER	Eliminar un usuario ya eliminado	`c> USER DOES NOT EXIST`	Correcto
UNREGISTER	Eliminar un nombre que nunca existió	`c> USER DOES NOT EXIST`	Correcto
REGISTER	Re-registrar el mismo nombre tras UNREGISTER	`c> REGISTER OK`	Correcto
CONNECT	Conectar un usuario registrado	`c> CONNECT OK`, hilo de escucha activo	Correcto
CONNECT	Conectar un usuario no registrado	`c> CONNECT FAIL, USER DOES NOT EXIST`	Correcto
CONNECT	Segundo CONNECT desde el mismo proceso cliente	`c> CONNECT FAIL, USER ALREADY CONNECTED`	Correcto
CONNECT	Conectar usuario ya conectado desde otro proceso	`c> USER ALREADY CONNECTED`	Correcto
CONNECT	Tres usuarios distintos conectados simultáneamente	Sin conflictos ni errores en ninguno	Correcto
DISCONNECT	Desconectar un usuario conectado	`c> DISCONNECT OK`, `_connected_user` puesto a `None`	Correcto
DISCONNECT	Desconectar un usuario que no está conectado	`c> DISCONNECT FAIL, USER NOT CONNECTED`	Correcto
DISCONNECT	Petición desde IP diferente a la del CONNECT	`c> DISCONNECT FAIL` (servidor rechaza por IP)	Correcto
DISCONNECT	Reconectar tras disconnect y recibir mensajes de nuevo	`c> CONNECT OK`, hilo de escucha funcional	Correcto

USERS	Listar con 3 usuarios conectados	`CONNECTED USERS (3 users connected)` con IP y puerto de cada uno	Correcto
USERS	La lista excluye usuarios desconectados y dados de baja	Solo aparecen los actualmente conectados	Correcto
USERS	Conteo correcto tras desconectar un usuario	`CONNECTED USERS (2 users connected)`	Correcto
USERS	USERS desde cliente no conectado	`c> CONNECTED USERS FAIL`	Correcto
SEND	Enviar mensaje a usuario online; primer mensaje del receptor	`c> SEND OK - MESSAGE 1`	Correcto
SEND	Receptor recibe el mensaje en su consola	`c> MESSAGE 1 FROM alice` + texto del mensaje	Correcto
SEND	Emisor recibe ACK de entrega	`c> SEND MESSAGE 1 OK`	Correcto
SEND	Segundo mensaje al mismo receptor → ID incremental	`c> SEND OK - MESSAGE 2`	Correcto
SEND	Tercer mensaje → ID sigue la secuencia	`c> SEND OK - MESSAGE 3`	Correcto
SEND	Enviar a un usuario inexistente	`c> SEND FAIL, USER DOES NOT EXIST`	Correcto
SEND	Enviar sin haber hecho CONNECT	`c> SEND FAIL, USER NOT CONNECTED`	Correcto
SEND	Autoenvío (usuario se manda mensaje a sí mismo)	`c> SEND OK`, el mismo usuario recibe el mensaje	Correcto
SEND	Enviar mensaje a usuario desconectado	`c> SEND OK` (almacenado en BD)	Correcto
SEND	3 mensajes offline de distintos emisores almacenados	Todos responden `SEND OK`	Correcto
SEND	Destinatario reconecta y recibe todos los pendientes	Los mensajes aparecen en consola al hacer CONNECT	Correcto
SEND	Emisores reciben sus ACKs en cuanto se entrega cada pendiente	`c> SEND MESSAGE N OK` para cada mensaje entregado	Correcto
SEND	Mensajes offline llegan en el mismo orden en que se enviaron	El mensaje 1 aparece antes que el 2, y el 2 antes que el 3 en consola	Correcto
SEND	Mensaje con caracteres UTF-8 (áéíóú, ñ, ü)	Texto recibido íntegramente sin corrupción	Correcto
SEND	Mensaje de 200 caracteres (cerca del límite MAX_MSG)	`c> SEND OK`, receptor recibe el texto completo	Correcto
SEND	10 envíos concurrentes de dos clientes simultáneamente	Los 10 envíos devuelven `SEND OK` sin deadlock	Correcto

SEND	IDs de mensaje independientes por receptor	El contador de charlie empieza en 1 aunque bob ya tenga IDs más altos	Correcto
SEND	Wrap-around: contador del receptor a `UINT_MAX`, siguiente mensaje	ID asignado = 1 (vuelve a empezar)	Correcto
SENDATTACH	Ruta de fichero sin barra inicial (no absoluta)	`c> SENDATTACH FAIL` (rechazado en cliente, sin conectar al servidor)	Correcto
SENDATTACH	Ruta relativa con subdirectorios	`c> SENDATTACH FAIL` (rechazado en cliente)	Correcto
SENDATTACH	Envío con adjunto a usuario online	`c> SENDATTACH OK - MESSAGE N`	Correcto
SENDATTACH	Receptor recibe el mensaje con nombre de fichero	`c> MESSAGE N FROM alice + FILE /ruta/fichero`	Correcto
SENDATTACH	Emisor recibe ACK con ID y nombre de fichero	`c> SEND MESSAGE N /ruta/fichero OK`	Correcto
SENDATTACH	Envío a usuario inexistente	`c> SENDATTACH FAIL, USER DOES NOT EXIST`	Correcto
SENDATTACH	Destinatario offline: mensaje con adjunto almacenado en BD	`c> SENDATTACH OK` (guardado con campo `filename`)	Correcto
SENDATTACH	Destinatario reconecta y recibe el adjunto pendiente	`FILE /ruta/fichero` aparece en la consola al reconectar	Correcto
SENDATTACH	Emisor recibe ACK tras la entrega del adjunto pendiente	`c> SEND MESSAGE N /ruta/fichero OK`	Correcto
GETFILE	Transferir fichero binario de 10.000 bytes (con bytes nulos y 0xFF)	`c> GETFILE OK`	Correcto
GETFILE	Tamaño del fichero recibido	10.000 bytes exactos	Correcto
GETFILE	Contenido del fichero recibido comparado byte a byte con el original	Contenido idéntico, sin ninguna corrupción	Correcto
GETFILE	GETFILE sin haber hecho CONNECT	`c> GETFILE FAIL`	Correcto
GETFILE	GETFILE de usuario que está desconectado	`c> FILE TRANSFER FAILED, user not connected.`	Correcto
Web Service	Endpoint `POST /quitar-espacios` responde	HTTP 200	Correcto
Web Service	`"hola mundo"` (doble espacio entre palabras)	`"hola mundo"`	Correcto
Web Service	`" al borde "` (espacios al inicio y final)	`"al borde"`	Correcto
Web Service	`"sola"` (sin espacios redundantes)	`"sola"` (sin cambios)	Correcto

Web Service	SEND con texto `"muchos espacios aqui"` con servicio activo	Receptor recibe `"muchos espacios aqui"` (normalizado)	Correcto
Web Service	SEND con Flask caído (fallback silencioso)	`c> SEND OK`, receptor recibe el mensaje sin normalizar	Correcto
RPC	Servidor RPC arranca y se registra en rpcbind	Proceso activo, no termina inmediatamente	Correcto
RPC	REGISTER con `LOG_RPC_IP` definida	Log RPC contiene la línea `"rpcalice REGISTER"`	Correcto
RPC	CONNECT con `LOG_RPC_IP` definida	Log RPC contiene `"rpcalice CONNECT"`	Correcto
RPC	SEND con `LOG_RPC_IP` definida	Log RPC contiene `"rpcalice SEND"`	Correcto
RPC	USERS con `LOG_RPC_IP` definida	Log RPC contiene `"rpcalice USERS"`	Correcto
RPC	DISCONNECT con `LOG_RPC_IP` definida	Log RPC contiene `"rpcalice DISCONNECT"`	Correcto
RPC	UNREGISTER con `LOG_RPC_IP` definida	Log RPC contiene `"rpcalice UNREGISTER"`	Correcto

4. Conclusiones

La parte más complicada de implementar fue el protocolo de transferencia de ficheros P2P. Al principio no teníamos claro cómo el cliente solicitante podía saber la IP y el puerto del otro cliente, hasta que nos dimos cuenta de que había que modificar USERS para que devolviera esa información junto con el nombre de usuario. Una vez entendido eso, el flujo en sí (el cliente solicitante manda un mensaje GETFILE directamente al socket de escucha del otro, que abre el fichero y lo envía en chunks) fue bastante directo de implementar.

Otro problema que tardamos bastante en encontrar fue un bug de doble incremento en la función que asigna identificadores a los mensajes. El contador del receptor se incrementaba dos veces: una en el `UPDATE` de SQLite y otra al calcular el nuevo ID en el código C, lo que producía IDs 2, 4, 6… en lugar de 1, 2, 3. Lo detectamos al escribir los tests de SENDATTACH, donde los IDs esperados no coincidían. La solución fue usar directamente el valor que ya devolvía el `SELECT` tras el primer `UPDATE`, sin volver a incrementar.

El servicio web fue lo más rápido de hacer, Flask es muy cómodo para esto. El único detalle que había que tener en cuenta era que escuchara en `0.0.0.0` y no solo en `localhost`, para que fuera accesible desde otros contenedores. En el cliente también fue importante el fallback silencioso: si el servicio no está disponible, el mensaje se manda igual sin espacios normalizados, para no bloquear el resto de la funcionalidad.

La parte del RPC fue interesante pero algo tediosa. El flujo de `rpcgen` genera stubs con código C antiguo que produce bastantes warnings con `-Wall -Wextra`, así que hubo que compilarlos por separado sin esas flags para no romper el build. La documentación de `libtirpc` tampoco es especialmente clara, y hubo que buscar ejemplos para entender cómo inicializar el cliente correctamente.

Por otro lado, dedicamos bastante tiempo a arreglar la comunicación entre diferentes clientes que no se estuviesen ejecutando en la misma máquina que el servidor. Finalmente optamos por probarlo con contenedores de Docker, y llegamos a la conclusión de que el problema era alguna regla de firewall, ya sea del router, o de las propias máquinas de Linux.

En general la práctica nos ha parecido bastante completa para entender cómo funciona un sistema distribuido real: concurrencia con threads, fallos parciales (qué pasa si el receptor se cae justo cuando le mandas un mensaje), coordinación entre varios procesos independientes con diferentes tecnologías de comunicación (sockets, HTTP, RPC) y persistencia para no perder mensajes. Todo eso junto hace que la práctica sea bastante representativa de un sistema de producción real, aunque simplificado.